



دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

دانشکده مهندسی کامپیوتر

پروژه نهایی اصول طراحی کامپایلر

AUJava to C Transpiler

استاد: دکتر ممتازی

ترم بهار ۱۴۰۴

تهیه و تنظیم تعریف پروژه: محمد حیدری راد و رضوان آفری

فهرست مطالب

۳	مقدمه
۳	معرفی AUJava
۵	تحلیل لغوی و تحلیل نحوی
۵	تحلیل معنایی
۷	تولیدکننده کد
۷	شرط و حلقه‌ها
۸	پیاده‌سازی کلاس‌ها
۱۰	کد سه آدرس
۱۰	ارث‌بری (امتیازی)
۱۳	پشتیبانی از field ها و method های static (امتیازی)
۱۳	شروط و محدودیت‌های برنامه‌های به زبان AUJava
۱۶	ملاحظات پیاده‌سازی
۱۶	نمره‌دهی

مقدمه

به پروژه نهایی درس اصول طراحی کامپایلر خوش آمدید! در این پروژه قصد داریم با استفاده از مطالب ارائه شده در درس، یک کامپایلر (در واقع یک transpiler) برای زبانی به نام AUJava بسازیم، که همانطور که حدس می زنید، یک زیرمجموعه از زبان Java است. زبان مقصد برای راحتی اشکال زدایی از پروژه و بالاتر بودن تسلط شما به این زبان نسبت به زبان های اسمبلی یا ماشین، زبان C انتخاب شده است.

معرفی AUJava

زبان AUJava که زبان مبدا مورد نظر است، از موارد زیر در زبان Java تشکیل شده:

- شی گرایی:
 - پشتیبانی از کلاس ها، فیلدها و متدها
 - ساخت شی و استفاده از آن
 - پشتیبانی از ارث بری (امتیازی)
 - پشتیبانی از متدها و فیلدهای static (امتیازی)
- ساختارهای کنترلی:
 - دستورات شرطی (if, else) برای کنترل جریان برنامه
 - حلقه while
 - پشتیبانی از break و continue
- موارد پایه ای:
 - پشتیبانی تنها از type های int و boolean
 - پشتیبانی از System.out.println برای چاپ ورودی های int و boolean
 - پشتیبانی از method call
 - پشتیبانی از عملیات های ریاضی +, -, *, /, % برای مقادیر عددی با رعایت اولویت مشابه زبان جاوا
 - پشتیبانی از تخصیص مقدار = به متغیرها و field ها
 - پشتیبانی از عملیات های جبر بولی &&, ||, ! برای مقادیر بولی
 - عملیات های مقایسه ای <, >, <=, >=, ==, != برای مقایسه
- برابری و نابرابری برای تمامی اشیا و متغیرهای primitive و کمتر و بیشتر و کمتر مساوی و بیشتر مساوی صرفا برای مقادیر عددی int

مواردی که پشتیبانی **نمیشوند**:

- دستورات switch case.
- حلقه‌های for و do-while
- دستورات increment و decrement (++, --)
- دستورات +=, -=, *=, /=, %
- کلاس‌های داخلی، interface ها، کلاس‌های بی نام و توابع بی نام (توابع لمبدا).
- محدودیت‌های دسترسی public, private, protected و package private.
- کلاس‌های abstract و interface ها.
- constructor های دلخواه (فرض می‌شود همه اشیا با استفاده از constructor پیشفرض قرار است ساخته شوند).

در زبان AUJava بر خلاف خود زبان Java، تمامی برنامه در یک فایل نوشته می‌شود. به همین دلیل تنها کلاس public موجود، همان کلاس Main است (که لزوماً نامش Main نیست) و بدون داشتن هیچ متد یا فیلد دیگری، صرفاً یک متد main دارد که به فرمت زبان جاوا، به صورت زیر نوشته می‌شود (تضمین می‌شود هیچوقت از ورودی args در برنامه استفاده نشود):

```
public class Main {  
    public static void main(String[] args) {  
  
    }  
}
```

باقی کلاس‌ها در برنامه می‌توانند بالا یا پایین این کلاس و به ترتیب دلخواه تعریف شوند، همینطور یک کلاس که بالاتر تعریف شده، می‌تواند از کلاسی که پایین‌تر تعریف شده استفاده کند، برای مثال چنین ساختاری پشتیبانی می‌شود:

```
class Student{  
    Teacher supervisor;  
}  
  
class Teacher{  
  
}
```

تحلیل لغوی و تحلیل نحوی

برای زبان توصیف شده، متناسب با ابزاری که برای توسعه پروژه استفاده می‌کنید، تحلیلگر لغوی و تحلیلگر نحوی بسازید. در صورت استفاده از ابزارهای آموزش داده شده در درس، صرفاً استفاده از **regular expression** ها برای تحلیلگر لغوی و **context free grammar** ها برای تحلیلگر نحوی برای ساخت تحلیلگرها کافی است. توجه داشته باشید که طراحی درست این بخش (مخصوصاً بخش تحلیلگر نحوی) در سطح سختی ادامه پروژه (تحلیل معنایی و تولید کد) نقش بسزایی خواهد داشت.

تحلیل معنایی

موارد معنایی برنامه که باید پوشش داده شوند:

- متغیر قبل از استفاده باید تعریف شده باشد و در **scope** مورد نظر قابل دسترسی باشد.
برای مثال کد زیر قابل قبول نیست، اگر نه در این **scope** و نه **scope** های بالاتر از آن، متغیر **x** قابل دسترسی نباشد:

```
int i = 100;  
System.out.println(x); // Error: 'x' is not defined!
```

- استفاده از متغیر درست از **scope** مورد نظر:

```
int i = 100;  
{  
    boolean i = true;  
    System.out.println(i); // Prints true, i is boolean  
}  
System.out.println(i); // Prints 100, i is integer
```

با وجود تعریف متغیر **i** در دو جا از برنامه، باید بسته به **scope**، متغیر درستی استفاده شود.

- چک کردن نوع:
استفاده از یک مقدار، متغیر یا فیلد، نیازمند متناسب بودن نوع آن است. برای مثال کد زیر قابل قبول نیست:

```
int i = true; // Error: Cannot assign boolean to int
```

- بررسی استفاده درست از متدها:

هنگام استفاده از یک `method`، باید بررسی شود که ورودی‌های آن با تعداد ورودی‌های ساختار تعریف آن متد یکسان هست یا خیر، و همینطور آیا نوع تک تک آن ورودی‌ها با نوع ورودی متناظر آن‌ها در تعریف متد سازگار هست یا خیر. برای مثال کد زیر قابل قبول نیست:

```
class A {
    void test(int a) {}
}

// in a method:
A a = new A();
a.test(true); // Error: Expected int but got boolean
a.test(100, 200); // Error: Expected 1 argument but got 2
```

- پشتیبانی از `this` و تحلیل معنایی متناسب آن:
کلمه کلیدی `this` برای اشاره به شی فعلی که `method` آن صدا زده شده به کار می‌رود و باید در زبان مقصد، به متغیر درستی نگاشت شود. نمونه استفاده از آن در `AUJava`:

```
class A {
    int i;
    void test() {
        i = 100; // Sets the instance variable i declared in the class
        int i = 200;
        System.out.println(i); // Prints 200
        System.out.println(this.i); // Prints 100
    }
}
```

- استفاده درست از `break` و `continue`:
باید چک شود که اصلاً در حال اجرای یک حلقه هستیم یا خیر، و در معنا نیز، صرفاً روی داخلی‌ترین حلقه‌ای که در آن هستیم تأثیر می‌گذارند. برای مثال:

```
class A {
    void test() {
        continue; // Error: Expected a loop scope
        while(true) break; // Ok: loop scope
    }
}
```

برای بخش تحلیل معنایی، چک کردن موارد بیان شده در کد ورودی، و همینطور نمایش خطای متناسب با آن در صورت نقض ضروری است. موارد زیر باید برای تحلیلگر معنایی رعایت شوند:

- استفاده از Symbol Table برای دنبال کردن متغیرها و مدیریت scope ها
- چک کردن type ها در expression ها، تخصیص مقادیرها، و صدا زدن متدها
- ارائه پیام خطای مناسب مرتبط با نقض اصول static semantic زبان AUJava، با اشاره به خط و کاراکتر مورد نظر تولید کننده خطا در کد برنامه ورودی

تولیدکننده کد

پس از حصول اطمینان درست بودن ظاهر برنامه به وسیله تحلیلگرهای لغوی و نحوی، و بررسی درست بودن معنای برنامه توسط تحلیلگر معنایی، کد برنامه ورودی را باید به زبان C تبدیل کنیم. برای این کار، نیاز است که ساختار برنامه به صورت Three Address Code (TAC) تولید شود و ساختارهای کد زبان AUJava به ساختارهای معادل در زبان C ترجمه شوند.

در ادامه، راهنمایی برای ترجمه ساختارهای AUJava به C ارائه می‌شود:

شرط و حلقه‌ها

این دستورات باید به جای دستور معادل، به goto و label ها در زبان C ترجمه شوند:

- استفاده از if, else در AUJava:

```
if (condition) {  
    // Then block  
    ...  
} else {  
    // Else block  
    ...  
}
```

کد معادل آن در C:

```
if (!condition) goto if_else;  
if_then:  
    // Then block  
    ...  
goto if_end;  
if_else:
```

```
// Else block
...
if_end:
```

- استفاده از while در AUJava:

```
while (condition) {
    // Loop body
    ...
}
```

کد معادل در زبان C:

```
loop_start:
if (!condition) goto loop_end;
    // Loop body
    ...
goto loop_start;
loop_end:
```

برای پشتیبانی از `continue` و `break`، یک راه این است که در یک `stack` دوتایی‌های `label`‌های شروع و پایان حلقه‌ها را نگه دارید، به طوری که هنگامی که وارد یک حلقه می‌شوید این دوتایی‌ها را بسازید و در این `stack` ذخیره و `push` کنید. حال اگر به هر کدام از دستورات `continue` یا `break` برخوردید، ابتدا چک کنید که این `stack` خالی است یا خیر. اگر خالی بود یعنی در هیچ حلقه ای نیستیم و برنامه خطا دارد، و اگر `stack` خالی نبود، دوتایی `label`‌های بالای `stack` را بررسی کنید، و اگر دستور `continue` بود به `label` شروع آن حلقه `goto` بزنید و اگر `break` بود به `label` پایان آن. این کار از این رو انجام می‌شود که `continue` و `break` بر روی درونی ترین حلقه‌ای که در آن هستیم تاثیر دارند.

پیاده‌سازی کلاس‌ها

در زبان AUJava، همانند زبان Java هر کلاس شامل `field`‌ها و `method`‌هایی است. برای هر کلاس، باید `struct` معادل آن در زبان C (به همان نام) در هنگام ترجمه ساخته شود. برای `field`‌ها، به سادگی از `field`‌های `struct` استفاده می‌کنیم، اما برای `method`‌ها، شرایط کمی پیچیده‌تر می‌شود و باید از `pointer to function`‌ها استفاده کنیم. مثال یک کلاس در AUJava:


```
class A {
    int a;
    int test(int d) {
        return a * d;
    }
}
```

مثال کد معادل در C و استفاده از pointer to function به عنوان کد معادل method:

```
struct A {
    int a;
    // Function pointer for the 'test' method
    int (*function_test)(void *, int);
};
```

تا اینجا صرفاً field و pointer to function را ساختیم. در تابع سازنده اشیا کلاس A، باید این تابع را مقدار دهی کنیم. برای این کار به شکل زیر عمل می‌کنیم:

ابتدا بدنه method را در یک تابع پیاده‌سازی می‌کنیم:

```
int A_function_test(void *caller, int d) {
    struct A* caller_A = (struct A *) caller;
    return caller_A->a * d;
}
```

سپس در تابع سازنده اشیا این struct، این تابع را به عنوان مقدار pointer to function متناظر آن در instance ساخته شده از struct تخصیص می‌دهیم:

```
struct A* new_A(){
    struct A *instanceA = malloc(sizeof(struct A));
    instanceA->function_test = A_function_test;

    return instanceA;
}
```

توجه داشته باشید که در زبان AUJava، متدها بر روی اشیا صدا زده می‌شوند، و شی‌ای که متد روی آن فراخوانی شده به وسیله this در دسترس است، اما در زبان C چنین امکانی نیست. از این رو، باید آن شی را به عنوان ورودی اول به تابع به عنوان caller ورودی داد.

حال برای استفاده از این تابع در کد، آن را به شکل زیر می‌توانیم فراخوانی کنیم:

```
int res = instanceA->function_test(instanceA, 100);
```

پیشنهاد می‌شود نام تابع سازنده اشیا را در زبان مقصد به تابع معادلی به شکل `new_{class_name}` ترجمه کنید که `class_name` نام کلاس شما است، و `return type` این تابع از نوع اشاره‌گری به `struct` معادل همان کلاس است.

کد سه آدرس

برای بدنه متدها، باید کد به شکل کد سه آدرس تولید شود، برای مثال اگر کد مبدا به زبان AUJava به صورت زیر باشد:

```
int x = 1 + 2 * 3;
```

کد معادل C آن باید به کدی مشابه زیر تولید شود:

```
int _t_1 = 1;
int _t_2 = 2;
int _t_3 = 3;
int _t_4 = _t_2 * _t_3;
int _t_5 = _t_1 + _t_4;
int x = _t_5;
```

ارث‌بری (امتیازی)

همانطور که می‌دانید، ارث‌بری یکی از ویژگی‌های اصلی برنامه نویسی شی‌گرا است، که به `class` ها اجازه می‌دهد که ویژگی‌هایی (از جمله `field` ها و `method` ها) را از کلاس والد خود ارث ببرند. از آنجا که C خود از ارث‌بری پشتیبانی نمی‌کند، باید ساختار معادلی برای پیاده‌سازی آن بسازیم. برای مثال، کد زیر را در AUJava در نظر بگیرید:

```
class A {
    int a;
}
class B extends A {
    int b;
}
```

چنین ساختاری در این زبان را می‌توان به شکل زیر در زبان C معادل‌سازی کرد:

```
struct A {
    int a;
};
struct B {
    struct A super; // Embedding structure A within B to simulate inheritance
    int b;
};
```

برای دسترسی به fieldها و method ها از کلاس‌های والد (یا والد آن والد و همینطور تا بالاتر) باید از فیلد super استفاده کرد، برای مثال:

```
struct B *instance = malloc(sizeof(struct B));
instance->super.a = 20; // Accessing field 'a' from the "inherited" part of B
```

خوبی استفاده از super به عنوان یک field (اولین field) در کد معادل C، این است که casting بین structهای والد و فرزند در زبان C را راحت می‌کند. برای این منظور، حتما باید super، اولین field ای باشد که در struct فرزند قرار دارد. علت این امر این است که چپش حافظه در struct والد، دقیقا ابتدای چپش حافظه struct فرزند قرار می‌گیرد و اگر instance به struct والدش cast شود، اشاره‌گر انگار به همان فیلد super اشاره دارد. همینطور اگر instance ای از struct والد به type فرزند cast شود، همچنان نقطه شروع آن‌ها در حافظه یکسان است و صرفا میزان حافظه‌ای که به آن اشاره می‌شود، گسترش یافته و شامل بقیه fieldهای struct فرزند نیز می‌شود. برای مثال:

```
// Example of casting
struct B *instanceB = malloc(sizeof(struct B));
struct A *instanceA = (struct A *)instanceB; // Cast B to A
// Accessing field 'a' directly through the casted instance
instanceA->a = 10;
```

کاربرد ارث‌بری صرفا در به ارث بردن fieldها نیست، بلکه نیاز به پشتیبانی method overriding نیز هست. چنین ویژگی‌ای در زبان‌های شی‌گرا امکان چندریختی را فراهم می‌کند. از آنجا که C یک زبان شی‌گرا نیست، از این ویژگی نیز پشتیبانی نمی‌کند، پس این ویژگی را نیز باید به نحوی از AUJava به کد معادل C تبدیل کنیم. برای مثال کد AUJava زیر را در نظر بگیرید:

```
class A {
    int a;
```

```

int test(int d) {
    return a * d;
}
}
class B extends A {
    int test(int d) {
        return a + d;
    }
}

```

در اینجا، کلاس B با ارث‌بری از کلاس A، متد test را override کرده. از آنجا که structها ذاتا شامل توابع نیستند، باید از pointer to function برای قرار دادن methodها در C استفاده کنیم. کد معادل استفاده از pointer to function در C:

```

struct A {
    int a;
    // Function pointer for the 'test' method
    int (*function_test)(void *, int);
};
struct B {
    struct A super;
    // 'super' structure to simulate inheritance
    // Function pointer for the 'test' method
    int (*function_test)(void *, int);
};

```

حال صرفا pointer to function ها را در structهای A و B قرار داده‌ایم. برای هر کدام از آنها، باید توابع مناسب پیاده‌سازی کرده و هنگام ساخت instance از این structها، این توابع را مقداردهی کنیم. توابع معادل:

```

int A_function_test(void *caller, int d) {
    struct A* caller_A = (struct A *) caller;
    return caller_A->a * d;
}
int B_function_test(void *caller, int d) {
    struct B* caller_B = (struct B *) caller;
    return caller_B->super.a + d;
}

```

مقداردهی pointer to function ها هنگام ساخت شی:

```
// in A constructor function (new_A):  
struct A *instanceA = malloc(sizeof(struct A));  
instanceA->function_test = A_function_test;  
  
// in B construction function (new_B):  
struct B *instanceB = malloc(sizeof(struct B));  
instanceB->function_test = B_function_test;  
instanceB->super.function_test = B_function_test; // Overriding the superclass method
```

حال با این شیوه overriding می‌توانیم به شیوه‌ای صحیح چندریختی در زبان AUJava داشته باشیم. برای استفاده از method می‌توان به شکل زیر عمل کرد تا آن‌ها را فراخوانی کرد:

```
int res = instanceB->function_test(instanceB, 100);
```

پشتیبانی از field ها و method های static (امتیازی)

این موارد از روی اشیا قابل دسترسی نیستند و صرفاً با استفاده از نام کلاسشان قابل استفاده هستند. همینطور در AUJava مشابه زبان Java در صورت پشتیبانی از ارث‌بری، کلاس فرزند نمی‌تواند با داشتن تابعی هم‌نام با تابع static در کلاس والد خود، آن را override کند (هر دو این تابع را با عملکرد متفاوت خواهند داشت). با توجه به اینکه تا اینجای کار ساختارهای مختلفی به شما نشان داده شده، ارائه ساختار مناسب برای تولید کد معادل این موارد به شما واگذار می‌شود. هرچند یک سر نخ می‌تواند تعریف آن‌ها به صورت global در زبان C و با استفاده از یک prefix نام کلاس در ابتدای نام آن‌ها باشد.

شروط و محدودیت‌های برنامه‌های به زبان AUJava

یک برنامه خوش ساختار در زبان AUJava قبل از ترجمه شدن به زبان C، باید دارای ویژگی‌های زیر باشد که باید در هنگام کامپایل شدن چک شود و در صورت نقض آن‌ها، پیام خطای مناسب به استفاده کننده از کامپایلر نشان داده شود:

- برنامه تنها یک نقطه شروع دارد.

برنامه‌های به زبان AUJava برای جلوگیری از هرگونه تناقض در شروع برنامه، باید دقیقاً یک تابع با

signature توابع main در زبان Java داشته باشند که به صورت زیر است:

```
public class Main {  
    public static void main(String[] args) {
```

```
}  
}
```

در صورت داشتن صفر یا بیش از یک تابع شروع `main`، باید از ترجمه جلوگیری شده و پیغام خطا چاپ شود.

- نام کلاس‌ها باید منحصر به فرد باشد.

در صورتی که کلاس‌هایی با نام یکسان در برنامه تعریف شده باشند، باید پیغام خطای مناسب چاپ شود، برای

مثال:

```
class A {}  
class A {} // Error: Class A is already defined
```

- قابلیت تعریف کلاس و متدها بدون ترتیب باید ممکن باشد.

ترتیب تعریف متدها و کلاس‌ها نباید روی معنای برنامه تاثیر بگذارد یا مانع از استفاده درست از آن‌ها شود،

برای مثال چنین تعریف‌هایی باید ممکن باشد:

- داشتن یک فیلد از یک کلاس که دیرتر تعریف شده:

```
class B { A a; }  
class A {}
```

- ارث‌بری از کلاسی که دیرتر تعریف شده (در صورت پشتیبانی از ارث‌بری):

```
class B extends A{}  
class A {}
```

- استفاده و ساخت اشیا از کلاسی که دیرتر تعریف شده:

```
class B{  
    A foo(){  
        A a = new A();  
        return a;  
    }  
}  
class A {}
```

- استفاده از متدی که دیرتر تعریف شده:

```
class B{  
    int foo(int n){  
        return bar(n);  
    }  
}
```

```
int bar(int m){
    return m;
}
}
```

- ترجمه متد چاپ `System.out.println`

در زبان AUJava برای پرینت کردن مقادیر عددی از این دستور استفاده می‌شود، در تحلیل معنایی باید بررسی شود که مقداری که به این تابع ورودی داده می‌شود حتماً از نوع `int` باشد (و در غیر این صورت باید پیغام خطا چاپ شود) و باید به شکل زیر به دستور معادل در C ترجمه شود:

کد AUJava:

```
System.out.println(arg);
```

کد معادل در زبان C:

```
printf("%d\n", arg);
```

- نادیده گرفتن کامنت‌ها در کد ورودی

کامپایلر باید کامنت‌ها را نادیده بگیرد و آن‌ها را به کد مقصد منتقل نکند. کامنت‌هایی که پشتیبانی می‌شوند به صورت تک خطی و چند خطی هستند:

```
// This is a single-line comment
/* This is a
multi-line comment */
```

- ارث‌بری حلقوی نباید وجود داشته باشد (در صورت پشتیبانی از ارث‌بری)

کمتر موقعی پیش می‌آید که ببینیم کسی چیزی را از خودش ارث ببرد (یا به عبارتی خودش پدرجد خودش باشد، البته فیلم `predestination` یک استثنا است). از این رو باید در تحلیل معنایی برنامه بررسی شود که این مورد نقض نشده است. برای مثال:

```
class A extends B {}
class B extends C {}
class C extends A {} // Error: Cyclic inheritance detected
```

ملاحظات پیاده‌سازی

- برای پیاده‌سازی پروژه، صرفاً استفاده از ابزارهای Lex, Yacc و Antlr که در درس و تمرین‌ها پوشش داده شده مجاز هستند (و همین‌طور پیاده‌سازی از صفر با زبان برنامه‌نویسی دلخواه). در صورت تمایل به استفاده از هر ابزار دیگری، قبل از انجام کار، با تدریس‌یارهای درس هماهنگی صورت گیرد. هرچند ابزار پیشنهادی تدریس‌یاران درس، ابزار Antlr است. در صورت استفاده از Antlr، مجاز به استفاده از هر زبانی که Antlr پشتیبانی می‌کند هستید، هرچند پیشنهاد تیم تدریس‌یاران زبان Java است.
- موارد تحویلی پروژه باید (حداقل) شامل موارد زیر باشد، حتی اگر لزوماً در ماژول‌های جداگانه پیاده‌سازی نشده باشند:
 - تحلیلگر لغوی (Lexer)
 - تحلیلگر نحوی (Parser)
 - تحلیلگر معنایی (Semantic Analyzer)
 - تولیدکننده کد (Code Generator)
 - گزارش کامل
 - مخزن **github** (یا هر مخزن عمومی دیگر) جهت بررسی روند انجام پروژه در هنگام ارائه به تدریس‌یاران

نمره‌دهی

- پروژه شامل ۲۰۰ نمره اجباری و (حداکثر) ۷۵ نمره امتیازی است که به شرح زیر تخصیص داده می‌شوند:
- موارد اجباری:
 - تحلیل لغوی درست و منطقی زبان مبدا (۲۵)
 - تحلیل نحوی درست و منطقی و کامل زبان مبدا (۴۵)
 - تحلیل معنایی درست و منطقی و ارائه خطای مناسب در شرایط مختلف به کاربر (۶۵)
 - تولید کد درست و معادل با کد مبدا (۶۵)
 - باید کدهای تولید شده توسط کامپایلر gcc قابل کامپایل باشند و به درستی اجرا شوند
 - موارد امتیازی:
 - پشتیبانی و تحلیل معنایی و تولید کد معادل درست و منطقی از ارث‌بری (۵۰)
 - پشتیبانی از static method ها و static field ها (۲۵)

توجه داشته باشید که کامپایلر شما باید با دریافت نام یک فایل در هنگام صدا زده شدن خودش در ترمینال، آن فایل را بخواند و در یک فایل با پسوند C (با نام دلخواه) نتیجه ترجمه را ذخیره کند و اگر کد ورودی خطا داشت، بدون تولید کد C، خطاها را در ترمینال چاپ کند.

توجه: در صورت هرگونه مشکل با تدریس‌یارهای درس از طریق پیام‌رسان "بله"، رضوان آفری با شناسه ghazal_afarin@ و محمد حیدری راد با شناسه rad81@ در ارتباط باشید.

پروژه برای طراحی و پیاده‌سازی برای گروه‌های حداکثر دوفره طراحی شده‌است. در صورت کشف شباهت بین پروژه گروه‌ها، به هیچ‌کدام از طرفین شباهت نمره تمرین تعلق نخواهد گرفت. همین‌طور در صورت مسلط نبودن به پروژه، نمره پروژه صفر در نظر گرفته خواهد شد.

فایل پروژه خود را به صورت zip شده با نام Proj_STUDENTNUM.zip در سامانه courses آپلود کنید، به صورتی که STUDENTNUM شماره دانشجویی (های شما به صورت جدا شده با _) است.

موفق باشید.